Practicum No.12 **Date:…….**

Stepper Motor using Matlab

I. Practical Significance

Stepper motors are widely used in precise motion control applications such as CNC machines, robotics, 3D printers, surveillance systems, and industrial automation. Interfacing a stepper motor with the AT89C51 microcontroller enables students to understand digital control of electromechanical systems, pulse sequence generation, and current amplification using driver ICs such as the ULN2003.This practicum demonstrates how digital signals from a microcontroller can be translated into controlled rotational motion through proper driving circuits. Students learn the fundamentals of stepper motor operation, excitation modes, torque considerations, and simulation testing using Proteus before hardware implementation. This forms the basis for real-world motion control systems and embedded automation solutions.

II. Competency and Industry-Specific Practical Skills

This practicum develops the following competencies aligned with industrial motion-control requirements:

Use the principles of computer architecture to optimize system performance.

a) Select the appropriate microcontroller and driver circuitry based on torque, current, and sequencing requirements (PDO).
b) Implement correct stepper motor excitation sequences (full-step, half-step) using AT89C51 programming (PDO).
c) Use driver ICs safely to protect low-power microcontrollers from overcurrent conditions (PDO).
d) Configure and simulate embedded motion-control systems using Proteus (PDO).
e) Troubleshoot miswiring, incorrect excitation sequences, and timing errors (CDO).

III. Relevant CO(s) *(As stated in the Course Structure)*

CO4: ***Use*** interfacing techniques for programmable peripheral devices to enhance input-output operations in computer systems.

(In this experiment, aligned to microcontroller–actuator interfacing fundamentals.)

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- **PrO:** ***Develop*** a system to display information representation for ***the given*** application

V. Related ADO(s) *(As stated in the Course Structure)*

a)**Follow** safe practices described for electronic laboratories.
b)**Handle** microcontroller kits and power supplies safely.
c)**Function** effectively as a team member.
d)**Maintain** clean wiring arrangements and proper housekeeping.
e)**Follow** ethical practices in reporting results.

VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

A stepper motor is an electromechanical device that converts electrical pulses into discrete mechanical movements. Each pulse rotates the shaft by a fixed angle known as the *step angle*. Two key concepts are:

1. Working Principle

A stepper motor consists of multiple electromagnetic coils arranged in phases. When these coils are energized in a defined sequence, they create magnetic fields that move the rotor in steps. Example excitation sequences:

- **Full-Step Sequence:** 4-step rotation
- **Half-Step Sequence:** 8-step rotation (smoother motion)

2. Need for Driver IC (ULN2003)

The AT89C51 cannot supply the current needed to energize the motor phases.
The **ULN2003 Darlington array**:

- Provides up to 500 mA per channel
- Protects the microcontroller
- Allows direct driving of stepper phases

3. Role of Proteus Simulation

Proteus allows:

- Visual verification of step sequence
- Observing stepper direction and speed
- Debugging without damaging hardware

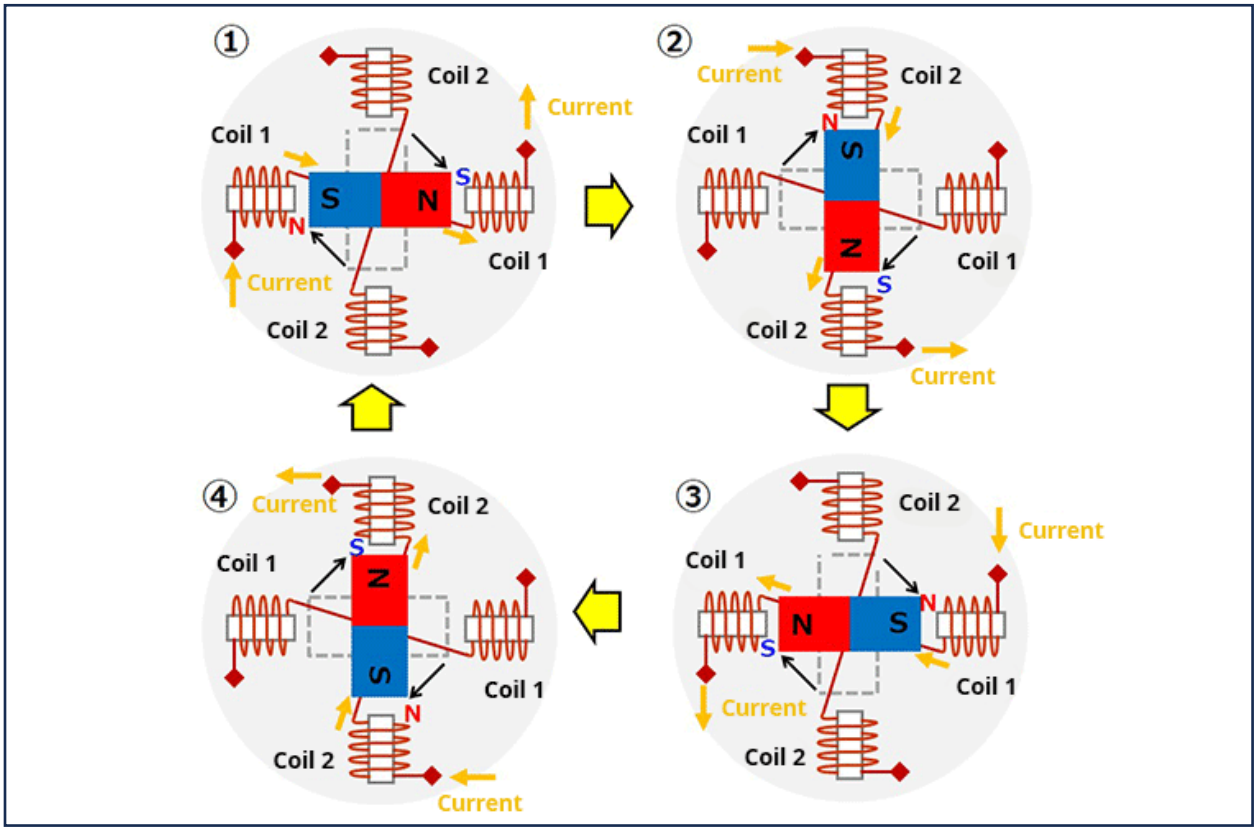
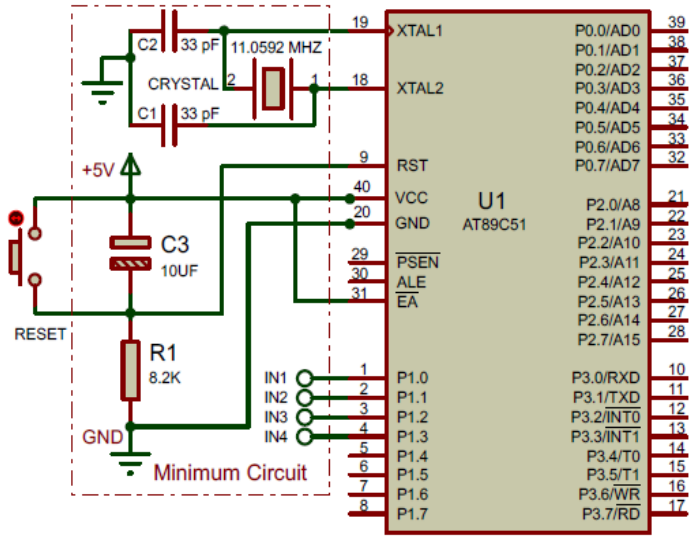
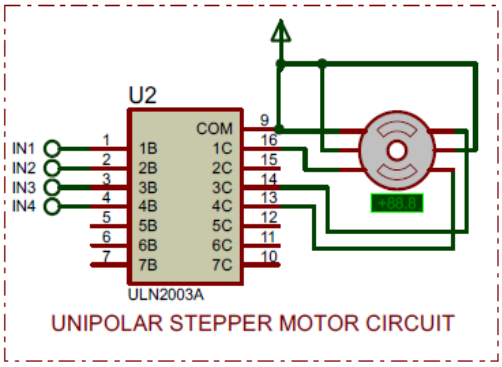


Figure 1 Stepper Motor

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart *(Include probable photos of experimental set-up.)*

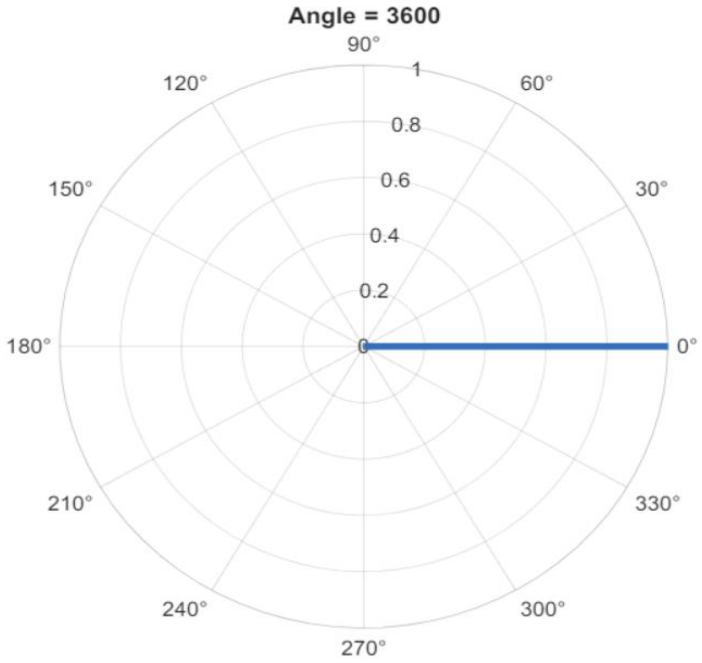


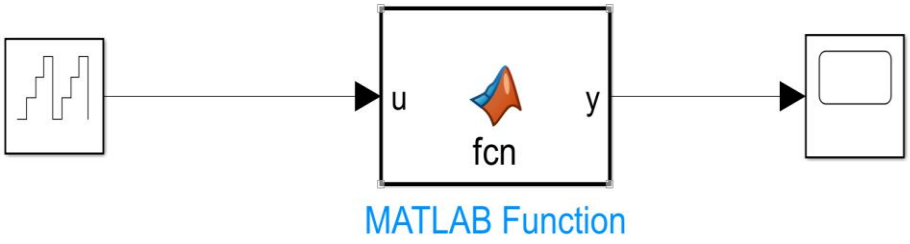


VIII. Matlab Simulation Code and Output

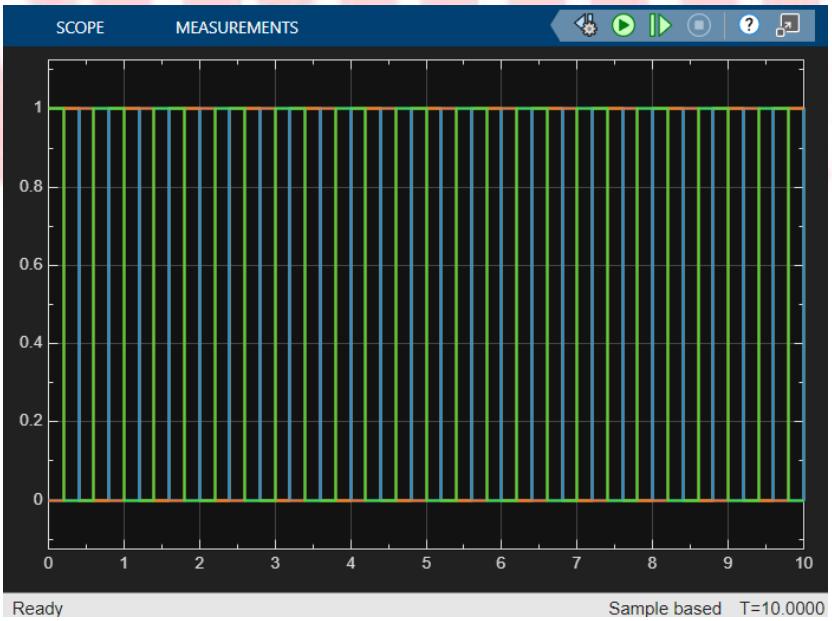
```
clc;
clear;
close all;
% Step sequence (Full Step)
sequence = [1 0 0 1;
1 0 1 0;
0 1 1 0;
0 1 0 1];

steps = size(sequence,1);
angle = 0;
delay = 0.5; % speed
figure;
for i = 1:10 % number of rotations
for j = 1:steps
% Increase angle
angle = angle + 90;
% Display sequence
disp(['Step ', num2str(j), ' -> ', num2str(sequence(j,:))]);
% Plot rotation
polarplot([0 deg2rad(angle)], [0 1], 'LineWidth', 3);
title(['Angle = ', num2str(angle)]);
pause(delay);
end
end
```

Output:





MATLAB Function



IX. Resources Required

S.No	Resource	Suggested Broad Specification	Quantity
1	Microcontroller	AT89C51 DIP-40	1
2	Stepper Motor	5-wire or 4-phase unipolar stepper (e.g., 28BYJ-48)	1
3	Driver IC	ULN2003 Darlington transistor array	1
4	Power Supply	5V or 12V depending on motor rating	1
5	Crystal Oscillator	11.0592 MHz + 33 pF capacitors (2 Nos.)	1 set
6	Reset Circuit	Push button, 10 kΩ resistor, 10 µF capacitor	1 set
7	Connecting Wires	For Proteus wiring	—
8	Proteus Software	Simulation of microcontroller and stepper	—
9	HEX File	Compiled program for sequence control	1 file
10	Decoupling Capacitors	0.1 µF (for MCU stability)	1–2

X. Safety Precautions

a) Ensure correct polarity while connecting power supplies to avoid damaging the microcontroller or motor.
b) Do not directly connect the motor to AT89C51 pins—always use ULN2003.
c) Verify voltage rating of the motor (5V/12V) before powering.
d) Avoid touching energized components in real hardware setups.
e) Keep wiring neat to avoid short circuits during testing.

XI. Procedure *(Self-Instructional)*

1. ***Study*** the circuit diagram showing connections between AT89C51, ULN2003, and the stepper motor.
2. ***Open*** Proteus and create a new schematic.
3. ***Place*** the **AT89C51**, **ULN2003**, and **Stepper Motor** from the library.
4. ***Wire*** the four IN pins of ULN2003 to AT89C51 Port pins (e.g., P2.0–P2.3).
5. ***Connect*** ULN2003 OUT pins to stepper motor coil wires.
6. ***Provide*** a separate power supply to the motor (5V or 12V as required).
7. ***Connect*** oscillator crystal (11.0592 MHz) and capacitors to AT89C51 XTAL1/XTAL2 pins.
8. ***Add*** reset circuitry to the RST pin.
9. ***Ensure*** all VCC and GND pins of components are correctly connected.
10. ***Load*** the **HEX file** into AT89C51 through *Edit Component → Program File*.
11. **Verify** pulse sequence in the assembly program (full-step or half-step).
12. ***Start*** simulation and observe the motor rotating in steps.
13. ***Adjust*** pulse delay in the program to vary motor speed.
14. Modify excitation sequence to reverse motor direction.
15. ***Validate*** that ULN2003 is correctly driving the motor coils by checking current flow.
16. ***Test*** different step modes and record observations.
17. ***Stop*** simulation, correct wiring if the motor vibrates but does not rotate.

18. ***Restart*** simulation and capture screenshots of successful rotation.
19. ***Experiment*** with half-step for smooth motion.
20. ***Document*** motor behaviour for different speeds.
21. ***Save*** simulation for future reference.
22. ***Record*** any anomalies such as motor stalling or missteps.
23. ***Validate*** direction control by modifying the pulse order.
24. ***Compare*** simulation results with theoretical step angle calculations.
25. ***Adjust*** delay loops to analyse torque-speed relationship.
26. ***Re-run*** simulation after minor code modifications.
27. ***Verify*** that the ULN2003 does not overheat during prolonged runs.
28. ***Test*** boundary conditions (very small/large delays).
29. ***Complete*** all required wiring checks to avoid shorts.
30. ***Prepare*** a detailed report including code, circuit, screenshots, and observations.

XII. Actual Resources Used

S. No.	Name of Resource	Broad Specifications		Quantity	Remarks (If any)
		Make	Details		
1.	MATLAB Software	MATLAB	R2025b	1	Used for simulation
2.	Stepper Motor Concept	-	Full step sequence Logic	1	Theoretical reference
3.	MATLAB Plot Tool	Built-in	Polar plotting (polarplot)	1	For visualization

XIII. Actual Procedure Followed

1. Opened MATLAB software and created a new script file.
2. Defined the full-step sequence for stepper motor operation.
3. Initialized variables such as angle and delay time.
4. Used nested loops to iterate through step sequences and rotations.
5. Incremented the angle by 90° for each step.
6. Displayed the step sequence in the command window.
7. Used polar plot to visualize the angular movement of the motor.
8. Added delay using pause () to simulate motor speed.
9. Executed the program and observed the rotational behaviour.

XIV. Results/Observations

1. The stepper motor simulation successfully showed rotation in discrete 90° steps using the full-step sequence.
2. The polar plot visually represented the angular movement, completing multiple rotations as per the loop.

XV. Interpretation of Results

1. The simulation demonstrates how a stepper motor rotates in fixed angular steps based on input sequences.

2. Each input sequence energizes coils in a specific pattern, producing controlled motion.
3. The 90° increment confirms correct implementation of the full-step mode.
4. The delay parameter directly affects the speed of rotation.

XVI. Conclusions

1. The MATLAB simulation effectively models the working of a stepper motor in full-step mode.
2. Controlled sequences result in precise angular displacement.
3. Visualization using polar plots helps in understanding motor rotation clearly.
4. This method can be extended for half-step and micro-stepping techniques for smoother motion.

XVII. Suggested Practicum Related Questions:

Note: *Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.*

1. Compare ULN2003-based driving with modern MOSFET-based stepper drivers in terms of torque, efficiency, and control flexibility.
2. Predict how advanced simulation tools and AI-based design assistants will transform embedded motion control design.
3. Interpret how load torque affects stepper motor acceleration and maximum speed.

XVIII References / Suggestions for further reading

1. AT89C51 Microcontroller Datasheet**,** Microchip.
2. ULN2003A Driver IC Datasheet, Texas Instruments.
3. Stepper Motor Basics – Application Note, STMicroelectronics.
4. Proteus Design Suite – Official Documentation, Labcenter.
5. Stepper Motor Interfacing with 8051 – Embedded Systems Tutorials.
6. NPTEL: Microprocessors and Microcontrollers (8051 Module).
7. 28BYJ-48 Stepper Motor Application Notes.

XIX Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the *'process-related skills/LOs'* (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and *'product-related skills/LOs'*

Note: The weightage for the ***process-related skills*** and ***product-related skills*** will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators		Maximum Marks	Marks Obtained
Process-related Skills: 18 Marks - 60%			
1	Handling of samples / Mechanical Tools / Microscope properly.	6	
2	Choice of relevant measurements.	5	
3	Follow safety precautions/protocols.	4	
4	Contribution as a team member (Rubric to be developed).	3	
Product-related Skills: 12 Marks - 40%			
1	Follow the principles underlying sample preparation.	3	
2	Follow the principles of the Interpretation of the images.	2	

Performance Indicators		Maximum Marks	Marks Obtained
3	Interpretation of Results.	3	
4	Conclusions.	2	
5	Practical related questions.	1	
6	Submitting the Journal on time.	1	
	Total	**30**	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	**Neavis Rishva. J**	**URK25CS1236**

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	**Product-Related Skills (12)**	**Total (30)**		

****